

Data Structures and Algorithms

Instructor: Dr. Hao Qin (秦浩)

E-mail: hao.qin@scupi.cn

My Office: Room 514, SCUPI (North)

Course Policies

- **General:**

- No cheating

- Each student is expected to do his/her own work. All code submitted should be written by you unless explicitly allowed by the course instructor.

- Attendance

- Students are expected to all synchronous lectures while tutorials are optional.

- Backup

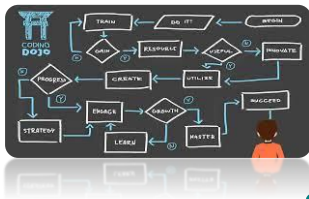
- Students are expected to have a backup disk (or storage somewhere) for every assignment they turn in. Data loss is NOT excusable.

What You'll Learn



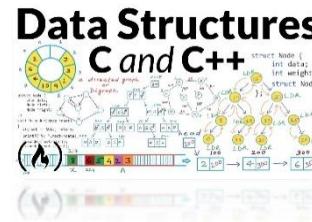
Advanced C++ Topics

Object Oriented Programming and C++ language features



Algorithms

The most useful algorithms (e.g., sorting, searching)



Data Structures

The most useful data structures (e.g., lists, trees, hash tables)



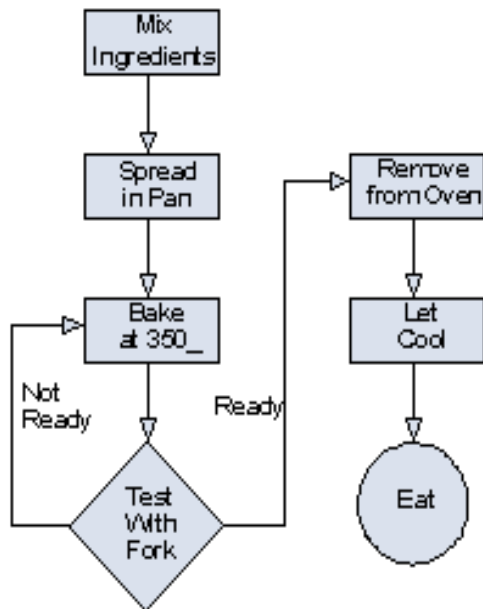
Building Big Programs

How to write large programs (1000+ lines)

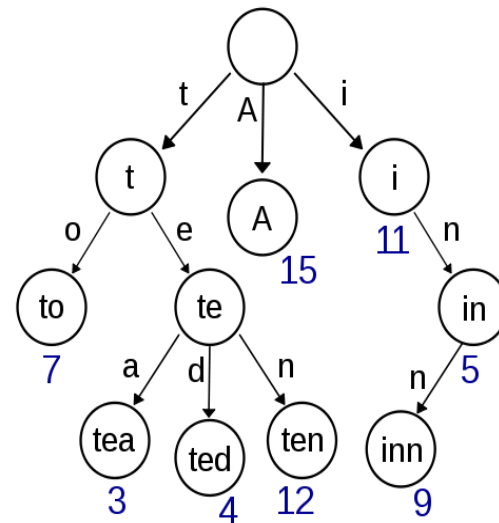


Let's learn about...

Algorithms



Data Structures



What is an Algorithm



An **algorithm** is a set of **instructions/steps** that solves a particular problem.

Each algorithm operates on **input data**.



42

Each algorithm produces an **output result**.

Each algorithm can be classified by **how long it takes to run** on a particular input.



Each algorithm can be classified by **the quality of its results**.

Algorithm Comparison

"Guess My Word"

Let's say you're building a word guessing game.

The user secretly
picks a random word
from the dictionary.



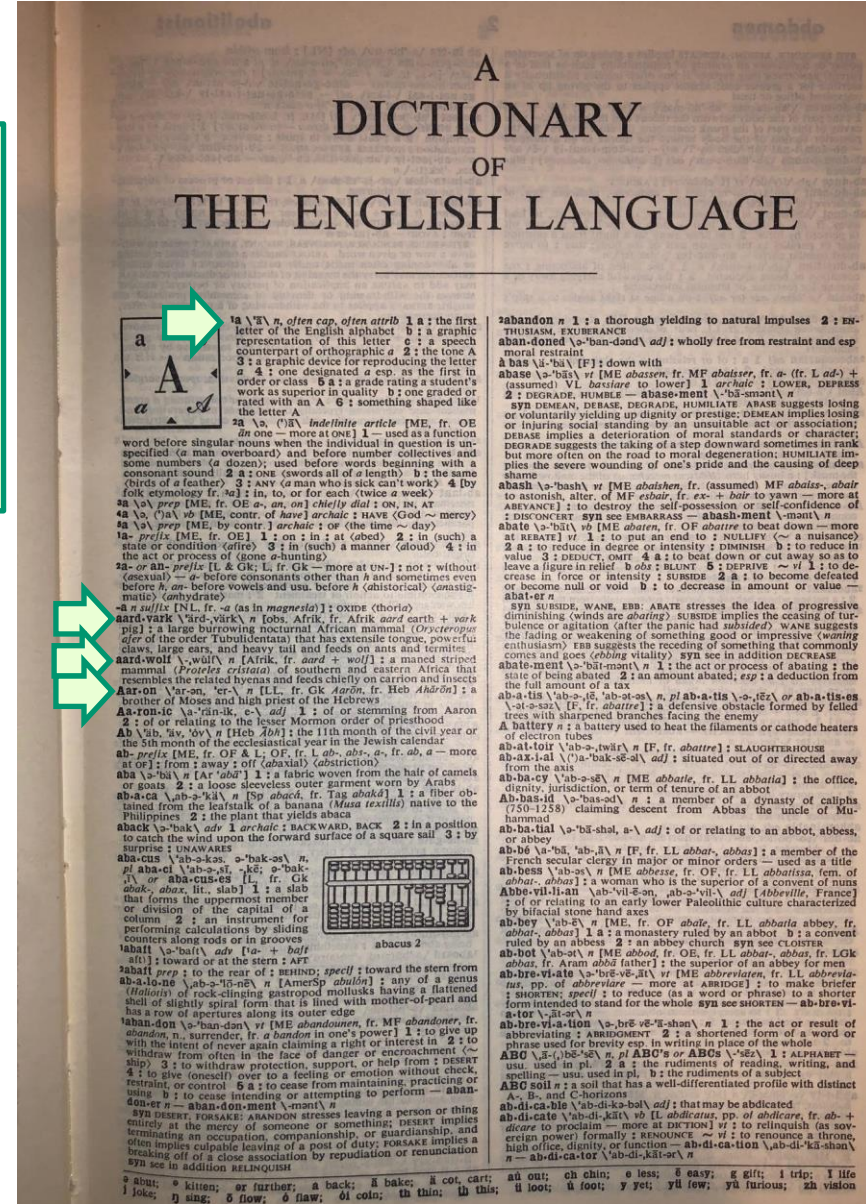
Our program then must
discover the word the user
picked as **quickly as possible**.

Let's consider **two** different algorithms...

Let's try a simple algorithm...

- Question:** If there are 100,000 total words in our dictionary, on average, how many guesses will our algorithm require?

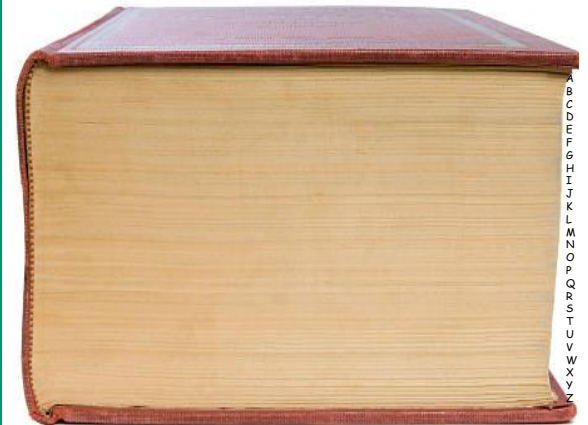
Ok, let's try it.



Algorithm #2: Binary Search

Alright, for our second strategy let's try a more intelligent approach called **binary search**:

1. Print: "Think of a random word"
2. While we haven't found the word:
 - a. Pick the middle word in the dictionary
 - b. Ask: "Does your word come before or after that word?"
 - c. If their word comes before, throw away the 2nd half of the dictionary
 - d. If their word comes after, throw away the 1st half of the dictionary



Question: If there are **100,000 total words** in our dictionary, on average, how many guesses will our Binary Search algorithm require?

Ok, let's try it.

Binary Search: How Many Guesses?

We keep on dividing our search area in half until we finally arrive at our word.

In the worst case, we must keep halving our search area until it contains just a single word – our word!



If our dictionary had 16 words, how many times would we need to halve it until we have just one word left?

16 8 4 2 1

It would take 4 steps

Ok, what if our dictionary had 131,072 words?

131072 65536 32768 16384 8192 4096 2048 1024 512 256 128 64 32 16 8 4 2 1

It would take just 17 steps!!! WOW!

Wow! That's Significant!

Linear search requires ~50,000 steps to guess a word from a 100k word dictionary.

But our binary search algorithm requires just 17 steps, on average, to guess the user's secret word!

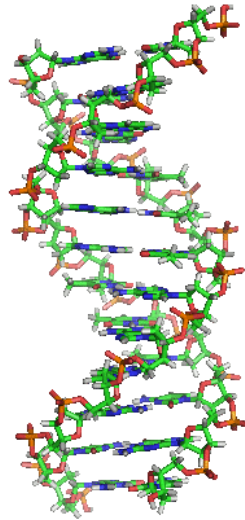


In this course, you'll learn:
all the major algorithms,
how to analyze them, and
how to pick the best one

Data Structures

 A **data structure** is the **set of variable(s)** that an algorithm uses to solve a problem.

Let's consider a data structure to efficiently store **millions of DNA sequences** so they can be searched.



DNA is made up of four bases:
Adenine, **Cytosine**, **Guanine** and **Thymine**

A Data Structure For DNA

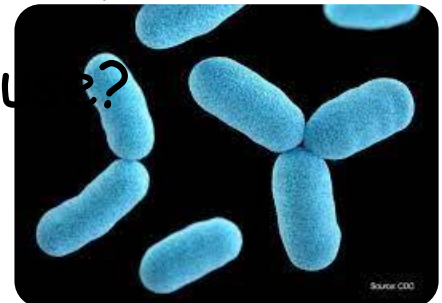
If I wanted to store millions of bacterial DNA sequences, each with 10 bases:

```
atggacatct  
acattaacga  
caccctcacc  
accgtagaat  
caccgccagct  
caccgaaatt
```

...

Corynebacterium
diphtheriae

What **simple data structure** might I use?



A Data Structure For DNA

Right! You could use a **sorted array** of strings!

```
string dna_seq[10000000];
```

```
dna_seq
```

[0]	acattaacga
[1]	accgtagaat
[2]	atggacatct
[3]	caccccagct
[4]	caccctcacc
[5]	caccgaaatt

... ..

Not bad. Given a new sequence, I could use **binary search** to determine if its is held in our data structure.

A Data Structure For DNA

Are there any **drawbacks** of the array data structure?

```
dna_seq
[0] acattaacga
[1] accgtagaat
[2] atggacatct
[3] caccgccagct
[4] caccctcacc
[5] caccgaaatt
... ..
```



atgaacatct

Right! If we wanted to add **even one new sequence**, we'd have to **move millions of items**, and that'd be slow.

A Data Structure For DNA

Are there any other **drawbacks** of the array data structure?

```
dna_seq
[0] acattaacga
[1] accgtagaat
[2] atggacatct
[3] caccgccagct
[4] caccctcacc
[5] caccgaaatt
... ..
```

Right! **Arrays** have a **fixed capacity**, so we're limited in how many DNA sequences we can add!

A Better Data Structure for DNA

Instead of an array, let's use a **Trie** data structure.

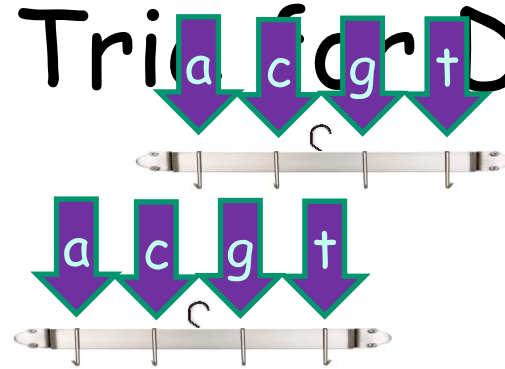
A Trie is just like a **balanced mobile**.



Let's see!

A Trick for DNA

Imagine each DNA sequence dangling from ten linked chains.



atggacatct



Now imagine a hanging bar with **four hooks**, one hook for **A**, one for **C**, one for **G** and one for **T**.



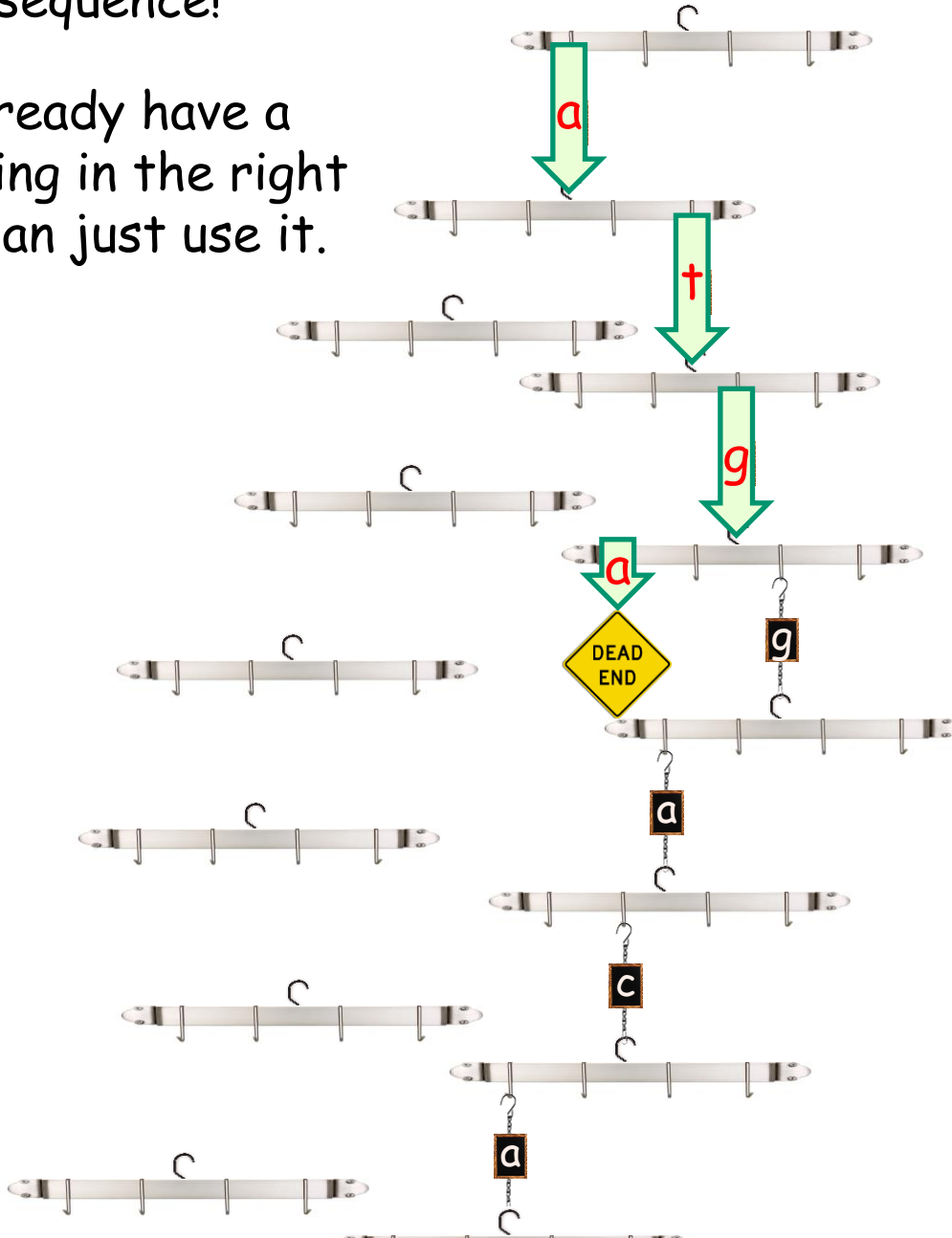
Starting with the first base of the sequence, **hang its chain** from the proper hook...

Add a new bar under the chain.

Repeat for all bases.

Ok, let's add a second DNA sequence!

If we already have a chain hanging in the right spot, we can just use it.



Now, to search if a sequence is in our Trie, we just start at the top and follow the chains!

Is "atgattacgt" in our Trie?

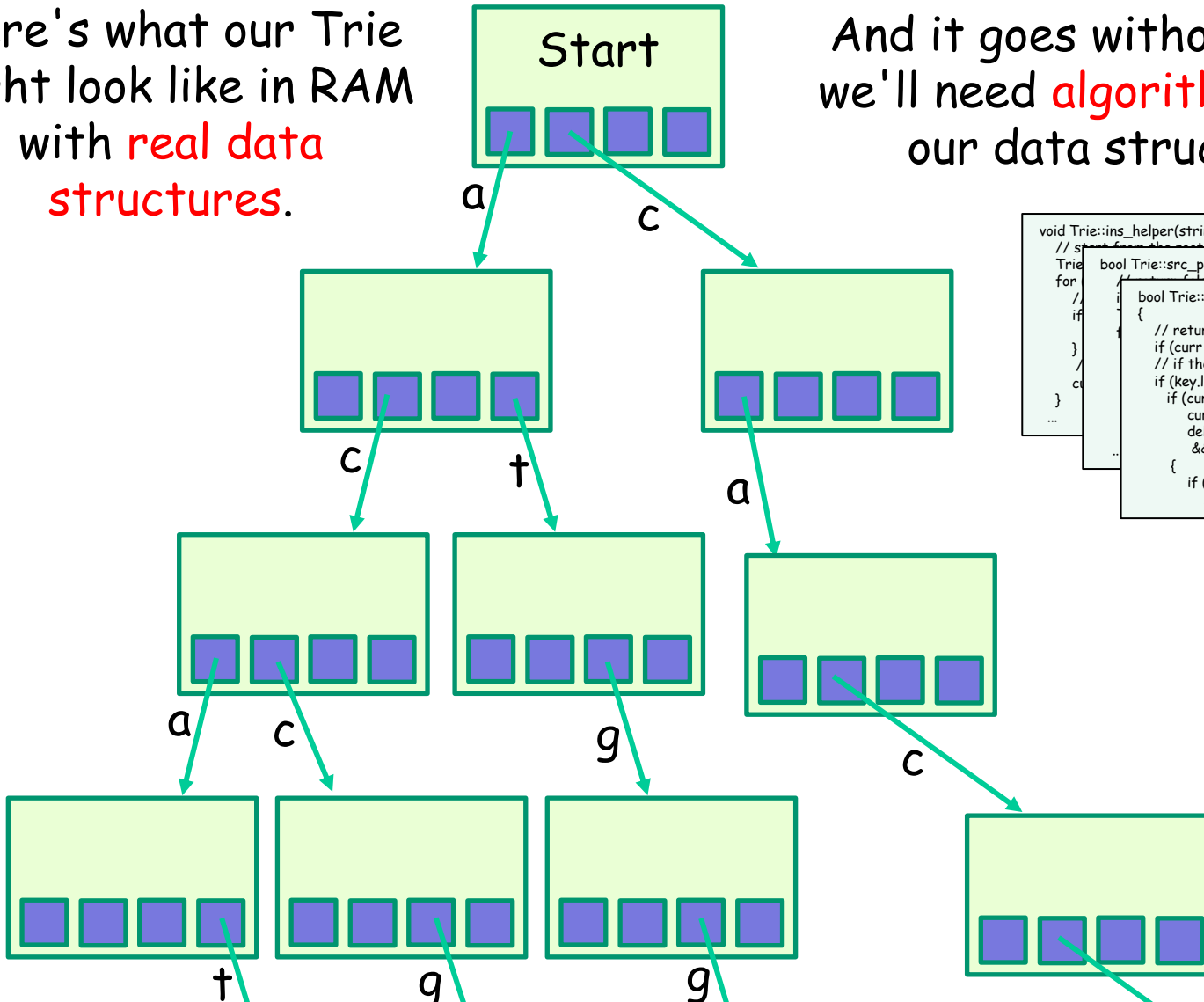


A Trie for DNA

Enough analogies with chains!

Here's what our Trie
might look like in RAM
with **real data**
structures.

And it goes without saying that we'll need **algorithms** to process our data structure too...



```
void Trie::ins_helper(string key) {
    // ...
    bool Trie::src_primitive(string key) {
        // ...
        bool Trie::del_nodes(Trie*& curr, string key)
        {
            // return if Trie is empty
            if (curr == nullptr) { return false; }
            // if the end of the key is not reached
            if (key.length() > 0) {
                if (curr != nullptr &&
                    curr->character[key[0]] != nullptr &&
                    deletion(curr->character[key[0]], key.substr(1))
                    && curr->isLeaf == false)
                {
                    if (!haveChildren(curr))
                        ...
                }
            }
        }
    }
}
// ...
}
```

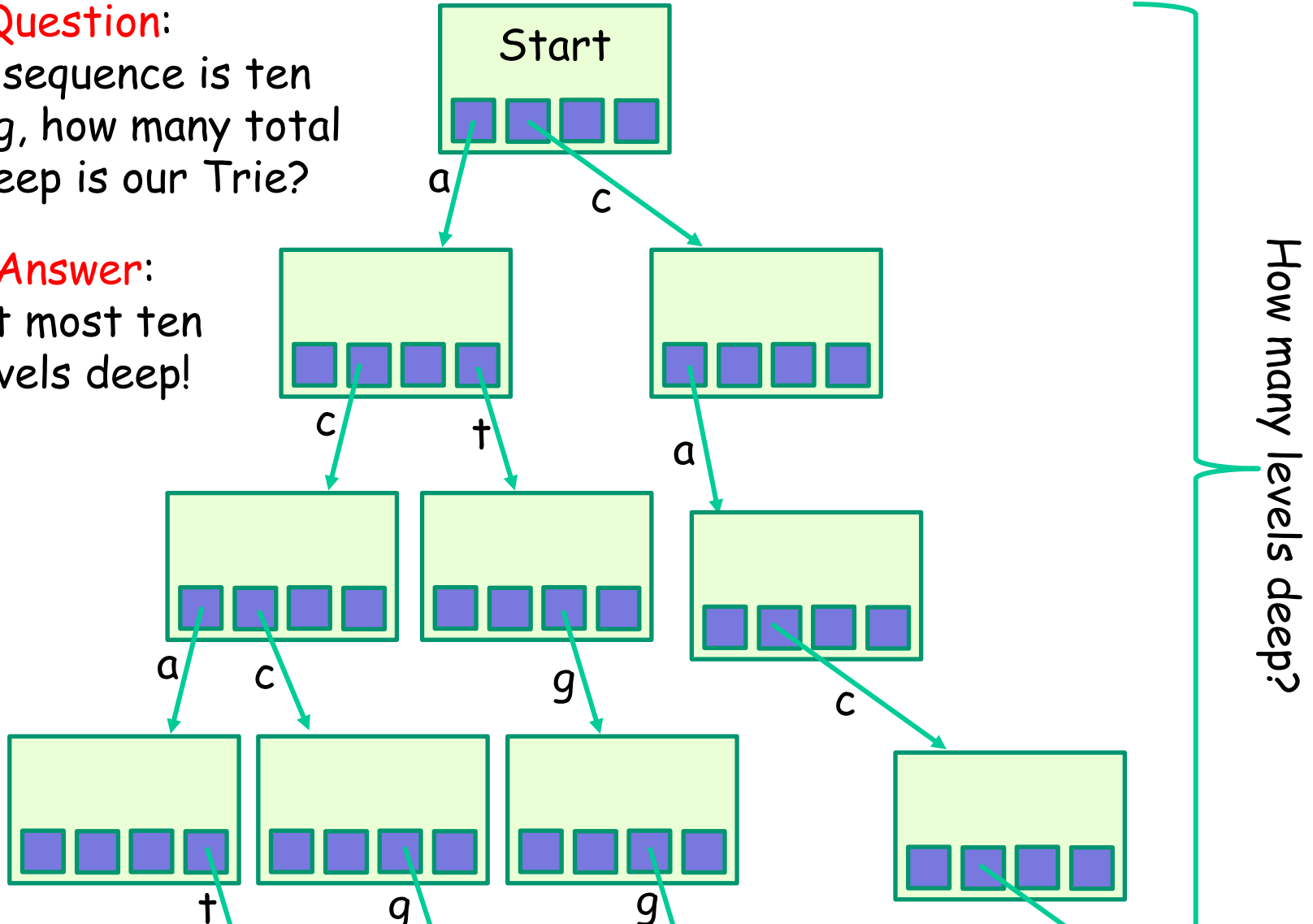
A Trie for DNA

Question:

If each sequence is ten bases long, how many total levels deep is our Trie?

Answer:

At most ten
levels deep!



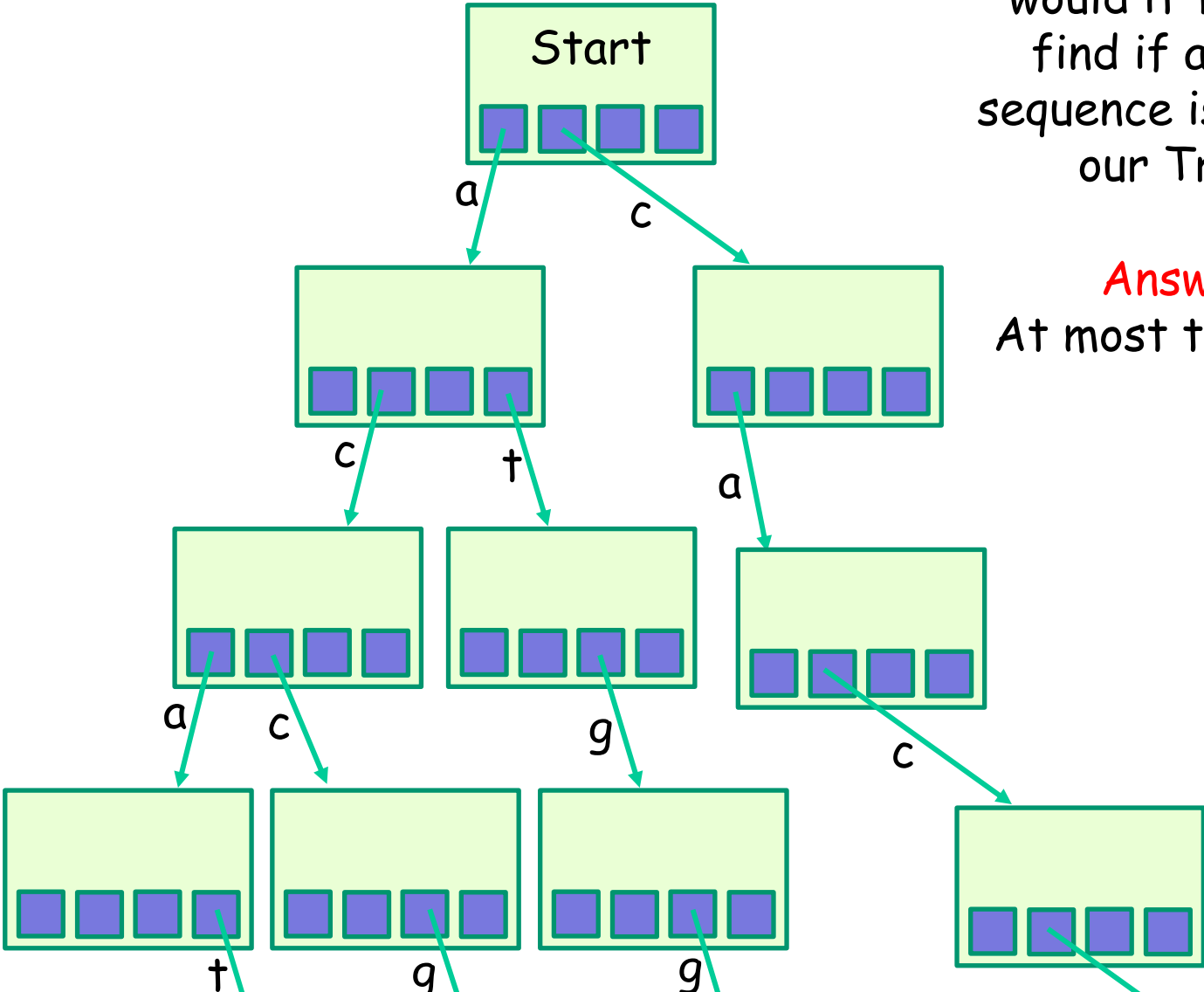
A Trie for DNA

Question:

How many steps
would it take to
find if a DNA
sequence is held in
our Trie?

Answer:

At most ten steps!



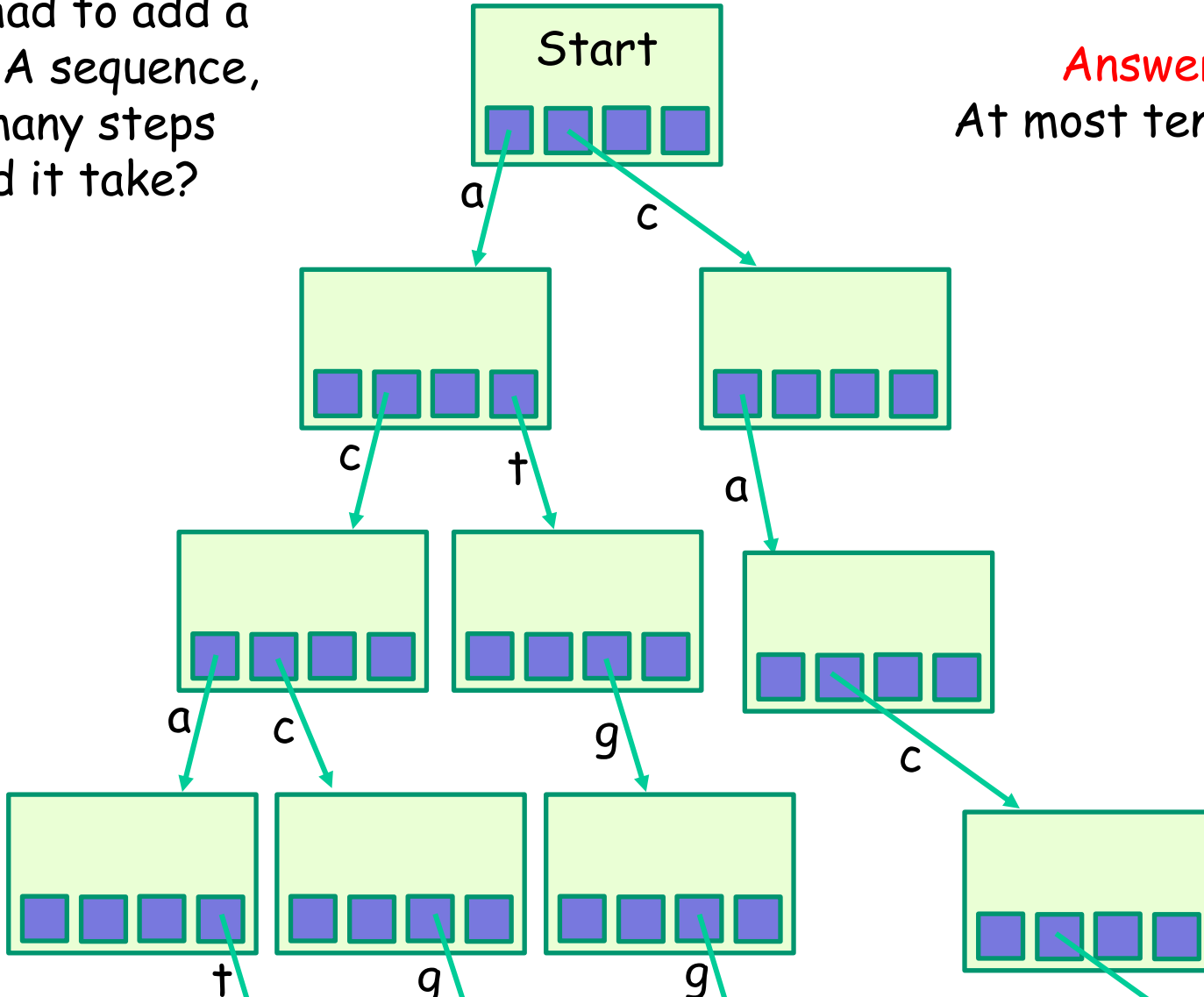
A Trie for DNA

Question:

If we had to add a new DNA sequence, how many steps would it take?

Answer:

At most ten steps!



Sorted Array vs. Trie

Sorted Array	Trie

Data Structures

As we can see, the right data structure can make your algorithms far more efficient!

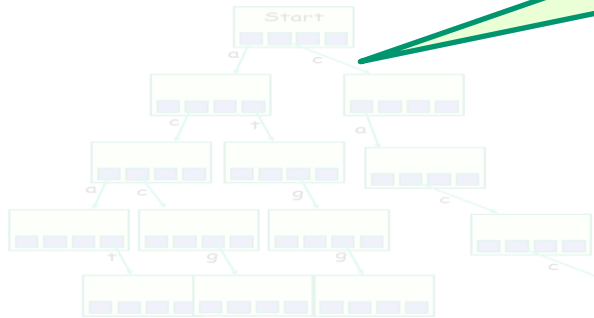
better

So in this course, we'll also learn all of the most efficient data structures!

Data Structures +

As we've seen, your data structures can get quite complicated.

Without having to understand any of the **confusing details**!



```
void Trie::ins_helper(string key) {
    // start from the root
    Trie* curr = root;
    for (int i = 0; i < key.length(); i++) {
        // create a new node if it doesn't exist
        if (curr->children[key[i]] == nullptr) {
            curr->children[key[i]] = new TrieNode();
        }
        curr = curr->children[key[i]];
    }
    curr->is_end = true;
}

bool Trie::isrc_primitive(string key) {
    // return true if the key is a primitive
    bool Trie::del_nodes(Trie*& curr, string key) {
        for (int i = 0; i < key.length(); i++) {
            // return if Trie is empty
            if (curr == nullptr) { return false; }
            // delete the node if it's not a primitive
            if (curr->children[key[i]] != nullptr) {
                del_nodes(curr->children[key[i]], key.substr(i+1));
            }
            curr->children[key[i]] = nullptr;
        }
        if (curr->is_end) { delete curr; }
        return true;
    }
}
```

Such a collection of simple functions is called an **"interface."**

If you gave your code to another programmer, they would have no idea how to use it!



So it always helps to have an interface.

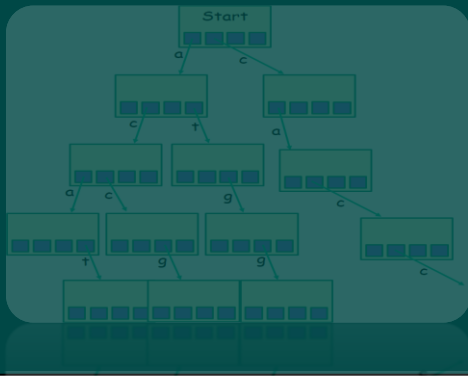
An **interface** lets any programmer use your code...

```
int main()
{
    addSequence("tgaccagact");
    if (findSequence("gcgttaacac") == true)
        cout << "This is a known DNA snippet!\n";
}
```

The Abstract Data Type (ADT)

In CS, we call a coordinated group of **data structures**, **algorithms** and **interface functions** an **Abstract Data Type**.

Abstract Data Type (for DNA searching)



+

```
void Trie::ins_helper(string key) {  
    // ...  
    bool Trie::src_primitive(string key) {  
        for (int i = 0; i < key.length(); i++) {  
            bool Trie::del_nodes(Trie* &curr, string key)  
            {  
                // return if Trie is empty  
                if (curr == nullptr) { return false; }  
                // if the end of the key is not reached  
                if (key.length() > 0) {  
                    if (curr != nullptr &&  
                        curr->character[key[0]] != nullptr &&  
                        deletion(curr->character[key[0]], key.substr(1))  
                        && curr->isLeaf == false) {  
                        if (!haveChildren(curr))  
                            ...  
                    }  
                }  
            }  
        }  
    }  
}
```

+

```
bool findSequence(string sequence);  
void addSequence(string sequence);  
void deleteSequence(string sequence);
```

In an ADT, the **data structures** and **algorithms** are **secret**.

The ADT provides an **interface** (a simple set of functions) to enable the rest of the program to use it.

Typically, we **build programs** from a **collection of ADTs**, each of which **solves a different sub-problem**.

ADTs in C++

In C++, we use **classes** to define ADTs in our programs!
Each C++ class holds **data structures**, **algorithms** and **interface functions**!

```
int main()
{
    DNADatabase d;

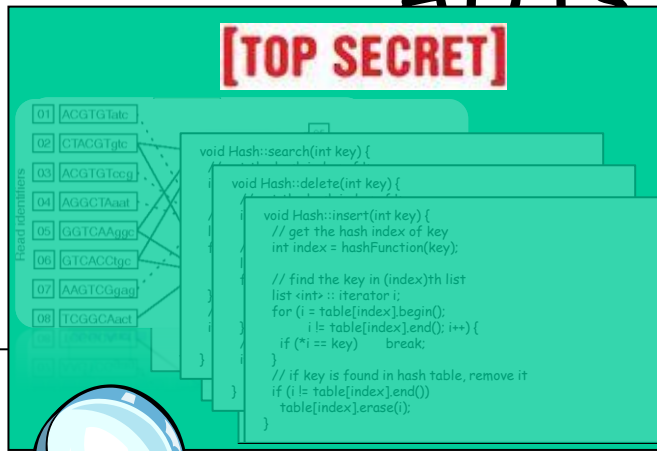
    d.addSequence("gagagtcaca");
    d.addSequence("tcaggacata");
    ...

    string dna_seq;
    cout << "Enter a 10-base sequence: ";
    cin >> dna_seq;
    if (d.findSequence(dna_seq) == true)
        cout << "This bacteria is known!";
}
```

```
// A C++ DNA sequencer class...
// (this is really an ADT!)

class DNADatabase
{
    public:
        // our interface functions go here
        void findSequence(...);
        void addSequence(...);
        ...
    private:
        // secret algorithms go here
        ...
        // secret data structures go here
        ...
};
```

ADTs in C++



```
int main()
```

```
{
```

```
    DNADatabase
```

```
    d.addSequence("gagagtcaca");
```

```
    d.addSequence("tcaggacata");
```

```
    ...
```

```
    string dna_seq;
```

```
    cout << "Enter a 10-base sequence: ";
```

```
    cin >> dna_seq;
```

```
    if (d.findSequence(dna_seq) == true)
```

```
        cout << "This bacteria is known!";
```

```
}
```

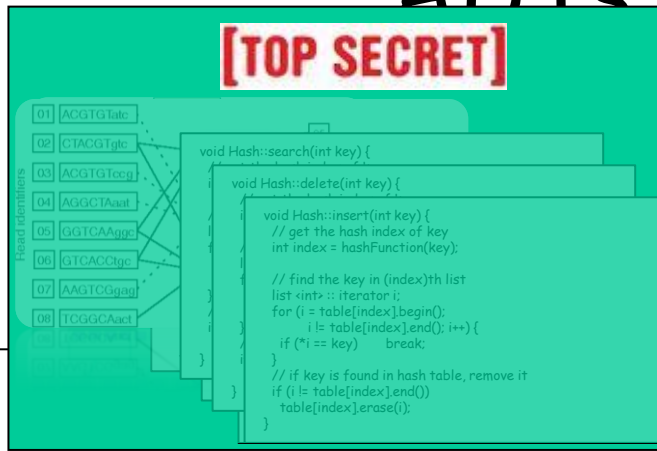
Now what if I wanted to **improve** my class's implementation?

Let's say I made a **radical change** to my **data structures** & **algorithms**...

Would the **user** need to **change** any part of their program?

No! Because these details are hidden from the rest of our code!

ADTs in C++



```
int main()  
{
```

```
    DNADatabase d;
```

```
    d.addSequence("gagagtcaca");
```

```
    d.addSequence("tcaggacata");
```

```
    ...
```

```
    string dna_seq;
```

```
    cout << "Enter a 10-base sequence: ";
```

```
    cin >> dna_seq;
```

```
    if (d.findSequence(dna_seq) == true)
```

```
        cout << "This bacteria is known!";
```

```
}
```

This is a **huge benefit** of
Abstract Data Types!

We can **break up our programs** into
small, **self-contained ADTs**...

And **combine these smaller parts**
together to **solve bigger problems**.

What is Object Oriented Programming?



Object Oriented Programming (OOP) is simply a programming model based on the Abstract Data Type (ADT) concept we just learned!

In OOP, programs are constructed from multiple self-contained **classes**.

Each class holds a set of **data structures** and **algorithms** - we then access the class using a **set of interface functions**!

Classes **talk to each other** only by using **public interface functions** - each class knows nothing about how the others work inside.

C++ Class Review

As we've seen, a "class" is a self-contained problem solver that contains:

- Data structures
- Algorithms
- Interface functions

Since you've probably forgotten everything about classes...

Let's do a quick review of classes by defining our own **Nerd** class!



Defining a New Class

```
class Nerd
{
    public:
        Nerd(int stink, int IQ) {
            myStinkiness = stink;
            myIQ = IQ;
        }
        void study(int hours) {
            myStinkiness += 3*hours;
            myIQ *= 1.01;
        }
        int getStinkyLevel() {
            int total_stink = myIQ * 10 +
                myStinkiness;
            return total_stink;
        }
    private:
        int myStinkiness, myIQ;
};
```

First, we write the outer shell of our **class** and give it a **name**.

Then we define our class's **public interface** functions...

Then we define our class's private **variables** and **functions**...

Our class defines an entirely new **data type**, like **int** or **float**, that we can now use in our program.

Alert: **Nerd** is **not a variable**!
It's a new C++ **data type**!

Using a New Class

Once we define a new **class**, like **Nerd**, we can use it to define variables like any traditional data type.

You typically define each new class in its own **.h file** ("header file") and put the file in the same folder as your **.cpp files**.

To use your new class, simply include its **header file** using "quotation marks"...

Just as **num_nerds** is an **integer** variable...

...we can use **variables** with it throughout your program.

When we define a new object, we can specify parameters to initialize it. These are called **objects** or **instances**.

nerd.h

```
class Nerd
{
public:
    Nerd(int stink, int IQ) {
        myStinkiness = stink;
        myIQ = IQ;
    }
    void study(int hours) {
        myStinkiness += 3*hours;
        myIQ *= 1.01;
    }
    ...
private:
    int myStinkiness, myIQ;
};
```

main.cpp

```
#include "nerd.h"

int main()
{
    int num_nerds = 1;
    Nerd david(30, 150);
    david.study(10);
}
```

Using a New Class

nerd.h

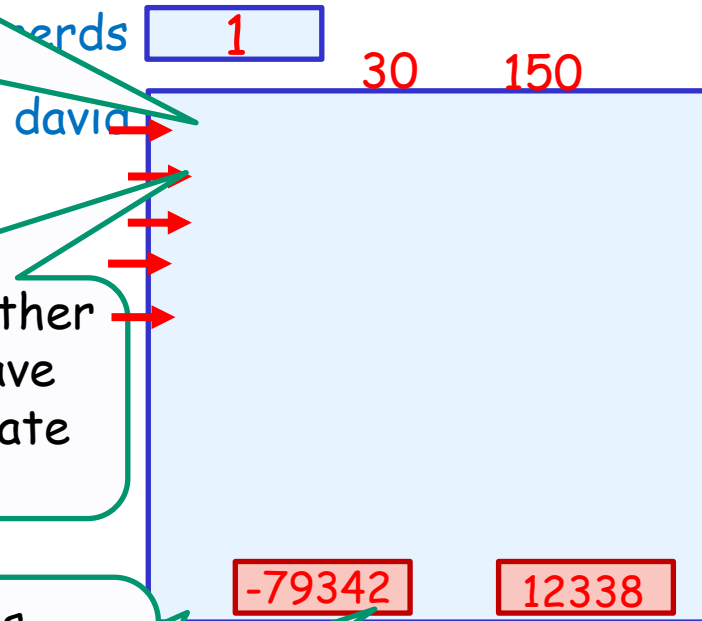
```
class Nerd
```

```
{  
public:  
    Nerd()  
    Nerd(int hours, int IQ);  
    void study(int hours);  
    ...  
private:  
    int myStinkiness, myIQ;  
};
```

Think of a **constructor** function as an **initializer**, e.g.:

```
Nerd david;  
david.initialize(30,150);
```

right, let's see our class in action!



Our constructor and other member functions have access to **david's** private member variables!

As soon as the variable is created, C++ immediately calls the **constructor** function inside the variable to initialize its state.

And so on...

(all **int's**, **doubles**) all start out with random values!

variables defined in your class!

main.cpp

```
#include "nerd.h"
```

```
int main()  
{
```

```
→ int num_nerds = 1;  
→ Nerd david(30, 150);
```

```
→ david.study(10);  
}
```

nerd.h

```
class Nerd
{
public:
    Nerd(int stink, int IQ) {
        myStinkiness = stink;
        myIQ = IQ;
    }
    int getStinkyLevel() {
        int total_stink = myIQ
            * 10 + myStinkiness;
        return total_stink;
    }
private:
    int myStinkiness, myIQ;
};
```

main.cpp

```
#include "nerd.h"
int main()
{
    int num_nerds = 1;
    Nerd david(30, 150);
    david.study(10);
}
```

Other Details

You use **member variables** to store **permanent attributes** of your class.

Stinkiness and **IQ** are inherent attributes of every **Nerd**.

So we make these member

On the other hand, we use **local variables** for temporary computations.

nerd.h

```
class Nerd
{
public:
    Nerd(int stink, int IQ) {
        myStinkiness = stink;
        myIQ = IQ;
    }
    void study(int hours) {
        myStinkiness += 3*hours;
        myIQ *= 1.01;
    }
    ...
private:
    int myStinkiness, myIQ;
};
```

So they can only be accessed by your class's member functions.

is class privacy!

All functions in the public section can be used by all parts of your program.

members are private.

main.cpp

```
#include "nerd.h"
int main()
{
    int num_nerds = 1;
    Nerd david(30, 150);

    cout << david.myStinkiness;
}
```

Code outside your class can't directly access them, however.

see our Nerd's function!

ERROR!